

WISEConv: A Worklist-Driven Masked-Convolution Runtime for GPUs

Anonymous Author(s)

Abstract

Masked convolution promises to cut the per-frame inference latency of vision models by computing only positions an upstream policy marks active, yet existing paths sacrifice either throughput or useful-compute ratio and fail to convert sparsity into proportional latency reduction. We present WISEConv, a masked-convolution runtime that derives an active-output worklist from the upstream mask and uses it to drive the dense tiled pipeline, securing both a high useful-compute ratio and high throughput by restricting computation to active positions while retaining the regular dense datapath. To amortize construction overhead and keep the compute pipeline fully utilized despite per-frame-varying worklist lengths, WISEConv co-designs construction and consumption: compatible operators reuse the constructed worklist without reconstruction, the worklist emission order preserves tile-local spatial reuse, and a data-driven adaptive decomposition adjusts parallel work to the activity level without host synchronization. Across four perception models and six GPU platforms (four discrete, two embedded), WISEConv reduces full-model latency by 22.1–46.5% relative to the fastest competing path and per-frame energy by 28.0–59.6% on the embedded platforms.

Keywords: Sparse convolution, dynamic spatial sparsity, GPU runtime, deep learning systems, efficient inference

1 Introduction

Dense convolution dominates the per-frame inference latency of many latency-critical vision tasks [2, 9, 37], yet on a given input much of that computation is spatially redundant. A common remedy is masked convolution: an upstream policy marks active positions, and the operator computes only those. The mask may derive from event-camera increments [7, 22], temporal differences [16, 27], or learned spatial gates [20, 34]. Regardless of source, it poses one operator-level problem: compute the convolution only at the active positions of the dense feature grid.

Dense GPU convolution obtains high throughput from tiled, coordinate-driven execution. Each tile enumerates a contiguous block of output coordinates; those coordinates fix each output position’s receptive field and write location, while the reduction over channels and kernel offsets follows a regular datapath. Neighboring coordinates share overlapping receptive fields, yielding predictable memory access and data reuse. However, unlike static sparsity in matrix multiplication, masked convolution selects positions at runtime per frame, requiring the operator to access scattered,

irregularly-overlapping neighborhoods and handle a varying workload, conflicting with tiled execution.

Existing masked-convolution systems preserve this regularity or eliminate wasted work, but not both. *Tile skipping* [7, 16, 27, 29] retains the dense pipeline and suppresses a tile only when all its outputs are inactive, so a single active position triggers computation for the entire tile. *Gather-scatter* [20, 34, 38] instead extracts active positions exactly, materializes receptive fields, and scatters the computation results to the dense grid, sacrificing regularity and incurring extraction, irregular access, and writeback overhead.

We characterize this trade-off along two axes (§2.2): *throughput* (operations per second) and *useful-compute ratio* (the fraction of issued operations that produce needed outputs). In our sparsest workload [34] (§5.2), the masks render 84.4% of the convolution work unnecessary, yet neither approach turns this into proportional latency reduction. Tile skipping [27] suffers a useful-compute ratio of only 49.1% and cuts latency by at most 40.6%. Gather-scatter drives the useful-compute ratio near one but sustains only 9.3% of tile skipping’s throughput, running 3.1× slower than the dense path.

Our key insight is that position-level selectivity does not require abandoning the dense convolution pipeline. A dense tile already derives its reads and writes from output coordinates, so replacing its contiguous coordinate block with an active-output worklist changes which outputs are computed without changing the per-position reduction, the dense tensor layout, or the weight layout. With this substitution, the operator runs as a dense tiled pipeline, pinning the useful-compute ratio near one without materializing input patches.

Translating this into latency reduction requires handling two challenges. First, *worklist-construction overhead*: construction scans the mask, so its cost scales with the layer’s spatial grid, not with the active count or channel widths, while the compute it enables shrinks with both. Construction’s share of latency therefore grows as activity falls and channels thin, and accumulates across operators when every layer rebuilds. Second, *end-to-end worklist efficiency*: the worklist’s order and length both affect its consumption throughput. An unordered worklist scatters the input neighborhoods that adjacent work touches, lowering locality; its length falls below the dense grid and shifts with layer and input, so no fixed tiling keeps the GPU full. Both effects sharpen as activity falls.

We present WISEConv (**W**orklist-**dr**Iven **ma**SkEd **Convo**lution), a masked-convolution runtime that fuses position-level selectivity into the dense tiled pipeline, securing both

a high useful-compute ratio and high throughput by co-designing worklist construction and consumption. Construction is cheap and infrequent: a single mask-space pass fuses output-mask propagation with per-tile compaction, operating on only mask and coordinate metadata; compatible operators reuse valid metadata instead of reconstructing it. Consumption is dense-like: construction emits contiguous per-tile segments ordered by the dense tile traversal, giving compute tile-local structure without global sorting. Compute then decomposes parallel work adaptively according to device, layer shape, and activity via a data-driven configuration that exploits the temporal correlation of streaming inputs, keeping the GPU fully utilized without host synchronization.

This paper makes the following contributions:

- We characterize masked-convolution latency along two axes, useful-compute ratio and throughput, exposing why existing work does not translate upstream sparsity into proportional latency reduction.
- We design WISEConv, which fuses position-level selectivity into the dense tiled convolution pipeline and co-designs worklist construction and consumption to secure both axes jointly.
- We evaluate WISEConv on four perception models across four discrete GPUs and two Jetson platforms. WISEConv reduces full-model latency relative to the fastest baseline by 34.2–46.5% on the discrete GPUs and 22.1–38.6% on the Jetson platforms, and cuts per-frame energy over dense convolution by 28.0–59.6% on the Jetson platforms.

2 Background and Motivation

2.1 Masked Convolution

Convolution runs inside the perception-action loop of many vision applications [2, 31, 35, 37, 41], where per-frame inference latency bounds system responsiveness [8–10, 17]. To cut this latency, masked convolution exploits dynamic spatial sparsity: an upstream policy emits a spatial mask, and the operator computes only the marked positions. The mask comes from sources including event-camera increments [7, 22], temporal residuals between video frames [3, 16, 26, 27, 36], learned input-dependent gates [12, 20, 34, 38]. The event and temporal policies mark positions that changed since the previous frame; the learned gates mark positions the task treats as relevant. These sources differ in sensing modality and policy, but all expose the same dynamic spatial sparsity on a dense 2D feature grid, so a single masked-convolution runtime serves them all.

Masked convolution keeps the tensors and arithmetic of dense convolution and restricts only which output coordinates it computes. It operates on an input feature tensor $\mathbf{x} \in \mathbb{R}^{H_{in} \times W_{in} \times C_{in}}$, a weight tensor $\mathbf{w} \in \mathbb{R}^{k \times k \times C_{in} \times C_{out}}$ of kernel size k , and an output $\mathbf{y} \in \mathbb{R}^{H_{out} \times W_{out} \times C_{out}}$ on a coordinate grid $\mathcal{G} = \{1, \dots, H_{out}W_{out}\}$, where each coordinate

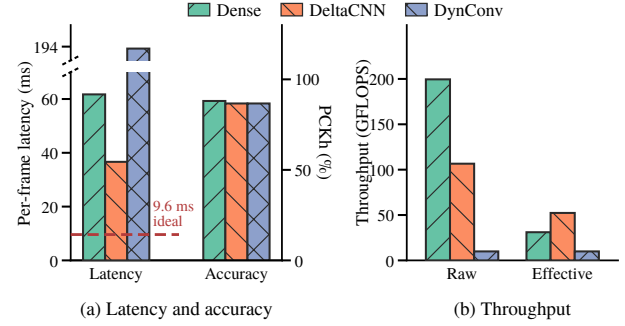


Figure 1. Statistics of DynConv [34], a pose estimator with learned spatial gating, on the MPII human-pose dataset [1] and Jetson AGX Orin, comparing the native cuDNN path (Dense) against the two sparse paths. (a) End-to-end average per-frame latency (left axis; the dashed line marks the dense latency scaled by the active ratio aggregated across layers) and PCKh accuracy (right axis; PCKh evaluates the percentage of keypoints localized within half the head size). The sparse paths apply the same mask, so their accuracy coincides. (b) Raw throughput T and effective throughput T_{eff} , aggregated across the model’s convolution layers. The dense useful-compute ratio implies the active ratio set by DynConv’s spatial gating ($\alpha \approx 15.6\%$).

$p \in \mathcal{G}$ has a $k \times k$ receptive field $\mathcal{N}(p)$ in $\mathbf{x}$. An upstream policy marks the coordinates to compute in an output mask $M_{out} \in \{0, 1\}^{H_{out} \times W_{out}}$. Spatial gating predicts M_{out} directly; event and temporal policies instead supply an input mask $M_{in} \in \{0, 1\}^{H_{in} \times W_{in}}$ whose active positions propagate their influence along the receptive field, so p stays active whenever any input in $\mathcal{N}(p)$ is active,

$$M_{out}[p] = \begin{cases} 1, & \exists r \in \mathcal{N}(p) \text{ s.t. } M_{in}[r] = 1, \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

M_{out} thus fixes which positions of $\mathbf{y}$ the operator computes, each by aggregating $\mathbf{x}$ over $\mathcal{N}(p)$ with the weights $\mathbf{w}$. The active ratio $\alpha = \|M_{out}\|_0 / |\mathcal{G}|$, the active fraction of the grid, is the intrinsic sparsity the policy exposes.

An execution path realizes this mask over a compute set C bounded by $\{p : M_{out}[p] = 1\} \subseteq C \subseteq \mathcal{G}$; for each $p \in C$ it computes

$$\mathbf{y}[p] = \sum_{i=1}^{k^2} \mathbf{w}_i \mathbf{x}[\mathcal{N}_i(p)], \quad (2)$$

where $\mathcal{N}_i(p)$ is the i -th position in $\mathcal{N}(p)$ and $\mathbf{w}_i \in \mathbb{R}^{C_{in} \times C_{out}}$ the corresponding weight slice. The path must compute every active coordinate to avoid dropping a requested output but may also compute inactive ones; dense convolution is the extreme $C = \mathcal{G}$. The policy fixes the accuracy and the required work, while C is determined by the path.

2.2 The Sparsity-to-Latency Gap

Existing systems execute a mask along one of two paths. *Tile skipping* [7, 16, 21, 26, 29, 36] inherits the dense pipeline’s tile-based computation, skipping a tile only when all its coordinates are inactive and otherwise computing the whole tile; DeltaCNN [27] fuses a selective path into the kernel, taken when a tile is estimated to be mostly empty, to cut wasted work. *Gather-scatter* [20, 38] instead extracts exactly the active coordinates, computes, and scatters them back, removing the inactive arithmetic but adding extraction, irregular access, and writeback overhead; DynConv [34] adopts a model structure suited to gather-scatter, where one gather feeds several consecutive layers before a single scatter, amortizing that overhead.

We measure each path by two quantities. Its useful-compute ratio is the fraction of computed coordinates that are active,

$$\eta = \frac{\|M_{out}\|_0}{|C|} \leq 1. \quad (3)$$

Its throughput is the rate at which it processes issued FLOPs,

$$T = \frac{2|C|k^2C_{in}C_{out}}{t}, \quad (4)$$

where t is the per-frame latency and the leading factor of 2 counts each multiply-accumulate as two FLOPs. For the required $2\|M_{out}\|_0k^2C_{in}C_{out}$ FLOPs, latency is set by the **effective throughput** $T_{eff} = \eta T$, the rate at which a path computes needed outputs.

Figure 1 traces this on Jetson AGX Orin for the sparsest workload in our evaluation. The policy marks 84.4% of the convolution work unnecessary, and the sparse paths discard it with negligible accuracy loss, so the remaining work sets a 9.6 ms latency floor (Fig. 1a, dashed). Neither path approaches it: tile skipping (DeltaCNN [27]) reaches 36.6 ms, still 3.8× the floor, and gather-scatter, even with DynConv built to amortize its overhead, runs at 193.5 ms, far slower than dense convolution itself. The gap traces to low effective throughput on both paths. Tile skipping holds throughput near half of dense but at a useful-compute ratio of only 49.1%; gather-scatter lifts the useful-compute ratio to one, yet extraction and data movement collapse its throughput to 5.0% of dense (Fig. 1b). Neither path efficiently converts the GPU’s capability into effective throughput for masked convolution.

2.3 Distinction from Related Sparse Execution

Related sparse-execution work differs from WISEConv’s masked convolution in representation, operator, sparsity type, or platform. 3D sparse convolution (SpConv v2 [30], TorchSparse++ [32, 33], Minuet [39], MinkowskiEngine [6], submanifold sparse convolution [15]) is built to organize computation over an unordered list of active voxels, constructing coordinate maps that route irregular input coordinates to outputs; its input is sparse by construction rather than a dense grid under a runtime mask, so it is related

infrastructure rather than a baseline. Sparse matrix multiplication (DTC-SpMM [11], MaxK-GNN [28], Sputnik [13]) reorganizes a sparse matrix operand so the dense datapath can consume it; its sparsity lives in the matrix structure of a different operator, not in a spatial mask over a convolution grid. Structured weight sparsity (2:4 tensor cores [24]) and sparse-format compilers (TACO [19], SparseTIR [40]) exploit a static structural sparsity fixed at compile time in the weights or a known format, rather than a mask discovered at runtime over the spatial grid. DPACS [14] resolves similar dynamic spatial sparsity in CNN feature maps, but on a custom accelerator instead of a commodity GPU.

3 WISEConv Design

WISEConv co-designs worklist construction and consumption so that selective execution retains the dense pipeline’s throughput while computing only active outputs. This section first states the design thesis and overviews the two-stage pipeline (§3.1), then describes construction (§3.2) and computation (§3.3).

3.1 Overview

As characterized in §2.2, masked-convolution latency is governed by effective throughput ηT , the product of the datapath’s raw throughput T and the useful-compute ratio η . No existing approach secures both axes: tile skipping preserves T but sacrifices η ; gather-scatter drives η near one but collapses T . The design thesis of WISEConv is that a worklist-driven operator can secure both jointly through co-design of construction and consumption.

WISEConv replaces the coordinate source of a dense tiled convolution with an active-output worklist derived from the upstream mask. The operator runs in two stages: a *construction stage* that converts the mask into a worklist of active output coordinates, and a *compute stage* that drives a tiled convolution from that worklist. Because our target is per-frame inference latency, every design decision is driven by minimizing single-frame overhead.

Construction targets η and its own amortization: it must produce a worklist cheaply, preserve spatial locality for compute, and avoid redundant rebuilds across compatible operators (§3.2). Compute targets T : it must consume the worklist at dense-pipeline throughput across varying lengths, without host synchronization or a fixed decomposition that underutilizes the device (§3.3).

3.2 Construction

Construction converts the upstream mask into a worklist before compute begins. The following subsections describe the two key mechanisms: tile-local compaction that produces spatially ordered segments without global sorting (§3.2.1), and a cross-operator reuse protocol that amortizes construction across compatible operator sequences (§3.2.2).

[TODO] WISEConv overview. Left: dense input feature tensor and input mask. Middle: tiled construction emits the output mask and compact worklist segments. Right: worklist coordinates replace the contiguous coordinate source of tiled convolution; only active outputs are written.

Figure 2. The WISEConv two-stage pipeline. Construction discovers active output coordinates and compacts each spatial tile into a worklist segment. Compute drives a tiled convolution from the worklist while retaining regular channel and kernel reduction.

3.2.1 Tile-Local Worklist Compaction. A naïve approach to worklist construction would scan the mask globally, collect all active coordinates, and then sort them to restore spatial locality. The sorting cost is $O(\alpha HW \log(\alpha HW))$ for an active ratio α , which adds a data-dependent overhead that partially offsets the compute savings the worklist enables. An alternative is to extract active positions along with their input neighborhoods (gather-scatter extraction), but this materializes $O(\alpha HW \cdot k^2 C_{in})$ data, far exceeding a metadata-only pass.

WISEConv avoids both costs by exploiting the tiled structure already present in the dense pipeline. Construction partitions the output grid into fixed spatial tiles and processes each tile independently. Within a tile, every output position checks whether any input in its $k \times k$ receptive field is active (the predicate of Eq. 1). Active positions are then compacted into a contiguous segment of the global worklist using warp-level ballot and atomic reservation: each warp counts its active lanes, the warp leader reserves a contiguous range in the worklist via an atomic counter, and each active lane writes its linearized output coordinate into the reserved range. The process simultaneously produces the output mask for downstream propagation.

The worklist buffer is pre-allocated at initialization to the output grid upper bound $|\mathcal{G}| = H_{out}W_{out}$ entries (one int32 coordinate each, totalling $4|\mathcal{G}|$ bytes). This avoids a two-pass scan (count then allocate) or a host-side allocation decision, both of which would require knowing the active count before construction writes. The additional memory per construction point equals the output feature map size along one channel, a fixed overhead independent of activity.

Algorithm 1 Tiled Worklist Construction

Require: Input mask M_{in} , output grid $\mathcal{G}$, tile size B

Ensure: Worklist $\mathcal{W}$, output mask M_{out} , active count n

```

1:  $n \leftarrow 0$ 
2: for each spatial tile  $T$  of  $B \times B$  positions in parallel do
3:   for each position  $p \in T$  in parallel within warp do
4:      $\text{active}(p) \leftarrow \exists r \in \mathcal{N}(p) : M_{in}[r] = 1$ 
5:      $M_{out}[p] \leftarrow \text{active}(p)$ 
6:   end for
7:    $\text{count} \leftarrow \text{number of active positions in warp \{ballot\}}$ 
8:    $\text{base} \leftarrow \text{ATOMICRESERVE}(n, \text{count}) \quad \{\text{warp leader}\}$ 
9:   for each active  $p$  at intra-warp rank  $r$  do
10:     $\mathcal{W}[\text{base} + r] \leftarrow \text{linearize}(p)$ 
11:   end for
12: end for
```

Algorithm 1 summarizes this procedure. The resulting worklist consists of per-tile contiguous segments whose internal order follows the spatial traversal within each tile. Cross-tile order is determined by the GPU's tile dispatch (approximately raster order) but is not globally sorted. Within each tile, threads are mapped to positions in row-major spatial order, so the ballot-based compaction preserves spatial adjacency in the output segment. Adjacent segment entries therefore address output positions whose $k \times k$ receptive fields overlap, enabling a single shared-memory load to service multiple outputs. Without this locality, each output would pull from a disjoint region of the feature tensor, exhausting cache capacity and replaying global memory loads.

Cost model. Construction visits every output position once to evaluate the receptive-field predicate, yielding a cost of $O(H_{out}W_{out} \cdot k^2)$. This cost is independent of the

channel dimensions C_{in} and C_{out} : construction operates on mask metadata alone and never touches the feature or weight tensors. For a fixed spatial shape, the cost is also independent of the active ratio, because every position must be checked regardless of the outcome. The formal consequence matches the intuition established in §1: construction's share of total operator latency grows as channels thin or activity falls, and accumulates linearly with layer count when every operator reconstructs. This motivates the reuse mechanism below.

3.2.2 Cross-Operator Worklist Reuse. Construction cost is channel-independent and precedes compute, so consecutive operators that each rebuild the worklist pay the same spatial-scan cost regardless of the compute they enable. In a model with many convolution layers at the same spatial resolution, this cost accumulates. Yet many of these rebuilds are redundant: any operator whose output grid coincides spatially with its input grid and whose active set is unchanged produces an identical worklist.

Compatible operators. A 1×1 convolution with unit stride, zero padding, and unit dilation maps each input position to exactly one output at the same spatial coordinate; it preserves both the grid geometry and the active set. Elementwise operators (activations, normalization) are pointwise by definition and satisfy the same conditions. Because such an operator must compute at exactly the positions where its input was written (the active set), and these positions are precisely those encoded in the upstream worklist, the worklist remains valid without modification.

Metadata protocol. WISEConv propagates worklist metadata across operators via a fixed tuple: the feature tensor, the spatial mask, the worklist buffer (an int32 coordinate array), and a scalar active count (a single int32 on device). When an operator is marked reuse-compatible at initialization, its forward path receives the upstream worklist buffer and active count directly and drives its compute kernel from them; no construction pass executes.

Activation and normalization layers participate by forwarding the received metadata unchanged: they compute their pointwise function at worklist positions and pass both the worklist buffer and active count to the next operator. Construction thus executes only at *segment boundaries* where the geometry changes (stride > 1 , kernel $k > 1$, or spatial resize); all operators within a compatible segment inherit the worklist from the segment's first construction.

Coverage. In a ResNet-style residual block, the typical sequence is: 3×3 conv (construction) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow 1 \times 1$ conv $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow 1 \times 1$ conv. A single construction pass serves the entire block. The same pattern holds for bottleneck blocks and inverted residuals, where the 3×3 depthwise convolution triggers construction and all surrounding 1×1 projections reuse it.

3.3 Computation

Construction delivers a worklist that pins η near one; the compute stage must now sustain high T while consuming it. Two properties of the worklist complicate this: its length is determined on device and varies per frame, and no single parallel decomposition keeps the GPU fully utilized across all activity levels. The following subsections address each in turn.

3.3.1 Persistent Worklist-Driven Convolution. In a standard dense convolution, the host knows the output size at launch time and dimensions the kernel grid accordingly. With a worklist, the output count becomes known only on the device after construction completes. A naïve approach would read the count back to the host before each launch, but this inserts a device-to-host synchronization per operator, stalling the streaming inference pipeline.

WISEConv eliminates this synchronization with a persistent kernel design. At module initialization, the runtime queries the device for the maximum number of thread blocks that can co-reside on each streaming multiprocessor (SM) given the kernel's resource usage (registers, shared memory). The launch grid is fixed to $\text{num_SMs} \times \text{blocks_per_SM}$ for the lifetime of the module. At kernel entry, thread 0 of each block reads the active count from device memory (written by the preceding construction). If the count is zero, all blocks return immediately; otherwise, each block iterates over work items in a grid-stride loop until the worklist is exhausted.

The compute kernel tiles work along three dimensions: B_M output positions per tile (drawn from the worklist), B_N output channels per tile, and B_K input channels per reduction step. When the worklist is short, output tiles alone may not fill the device; *split-K* partitions the channel reduction across S_k independent blocks that accumulate partial results via atomic writeback, trading additional parallelism for a per-position atomic cost. The total work is therefore $\lceil n/B_M \rceil \times \lceil C_{out}/B_N \rceil \times S_k$ items, distributed across the persistent grid.

Gather-at-load. For each output tile, the kernel reads output coordinates from the worklist, decomposes each into batch, row, and column indices, and computes the receptive-field base address. During the channel and kernel reduction, the kernel gathers input values directly from the dense feature tensor using these addresses (implicit GEMM), identical to how a dense tiled convolution accesses its inputs. No intermediate im2col buffer is materialized; the only additional cost per position is a few integer operations for index decomposition.

Algorithm 2 summarizes the persistent compute model. The structure mirrors a standard tiled implicit-GEMM convolution (outer loop over output $\times$ channel tiles, inner loop over spatial and channel reduction), with two differences: the outer loop is grid-stride rather than one-to-one with blocks,

Algorithm 2 Persistent Worklist-Driven Convolution

Require: Worklist $\mathcal{W}$, active count n , input $\mathbf{x}$, weights $\mathbf{w}$
Ensure: Output $\mathbf{y}$ (updated at worklist positions)

```

1:  $n \leftarrow \text{read active count from device memory}$ 
2:  $N_{\text{work}} \leftarrow \lceil n/B_M \rceil \times \lceil C_{\text{out}}/B_N \rceil \times S_k$ 
3: for tile = block_id, block_id + grid_size, ... <  $N_{\text{work}}$ 
   do
4:    $(b_m, b_n, s) \leftarrow \text{decompose tile}$ 
5:   Load worklist entries  $\mathcal{W}[b_m \cdot B_M \dots (b_m+1) \cdot B_M]$ 
6:   Decode each entry to  $(b, h_{\text{base}}, w_{\text{base}})$ 
7:   Initialize accumulator  $\mathbf{Y} \leftarrow \mathbf{0}_{B_M \times B_N}$ 
8:   for  $r = 0, \dots, k-1$ ;  $c = 0, \dots, k-1$  do
9:     for  $t = 0, B_K, \dots < C_{\text{in}}$  do
10:       $\mathbf{X} \leftarrow \text{gather from } \mathbf{x} \text{ at } (h_{\text{base}}+r, w_{\text{base}}+c, t)$ 
11:       $\mathbf{W} \leftarrow \text{load } \mathbf{w}[r, c, t:t+B_K, b_n]$ 
12:       $\mathbf{Y} \leftarrow \mathbf{Y} + \mathbf{X} \cdot \mathbf{W}$ 
13:     end for
14:   end for
15:   Scatter  $\mathbf{Y}$  to  $\mathbf{y}$  at worklist positions
16: end for

```

and the M-dimension indices come from the worklist rather than a contiguous range.

Properties. The persistent design ensures that the entire compute path operates without host involvement: the launch grid is fixed, the work bound is read on-device, and all output positions are determined by the worklist. The per-position reduction (lines 8–13) is identical to dense convolution: the same channel and spatial loops, the same memory access pattern within a tile, and the same arithmetic. Position-level selectivity changes only *which* positions are computed, not *how* each position is computed, preserving T .

The persistent grid eliminates host synchronization but still uses a fixed tile configuration. As the worklist shortens, this fixed decomposition leaves the device under-utilized; the next subsection introduces a data-driven adaptation that complements the persistent grid.

3.3.2 Adaptive Workload Decomposition. The tile configuration (B_M, B_N, S_k) determines how work is decomposed: a larger output tile amortizes shared-memory loads but produces fewer tiles from a short worklist, lowering occupancy; conversely, a higher split-K factor adds parallelism along the reduction dimension at the cost of atomic accumulation per position. The optimal tradeoff shifts with the worklist length. At low activity, few output tiles exist and split-K is needed to fill the device; at high activity, output tiles alone provide sufficient parallelism and split-K's atomic cost becomes a net loss. A single fixed configuration cannot serve both regimes without sacrificing T at one end.

Selecting the optimal configuration at runtime requires knowing the activity level, yet reading it from the device per layer would reintroduce the synchronization the persistent

Algorithm 3 Per-Frame Adaptive Configuration

Require: Ring buffer $\mathcal{R}$ (N_s slots), calibration table $\mathcal{L}$, convolution modules $\{m_i\}$

```

1: count  $\leftarrow \text{ASYNCPOLL}(\mathcal{R})$  {non-blocking event query}
2: if count is available then
3:    $\alpha \leftarrow \text{count} / |\mathcal{G}|$ 
4:    $\ell \leftarrow \text{NEARESTLEVEL}(\alpha)$ 
5:   for each convolution module  $m_i$  do
6:      $(B_M, B_N, S_k) \leftarrow \mathcal{L}[m_i.\text{shape}, \ell]$ 
7:      $m_i.\text{SETCONFIG}(B_M, B_N, S_k)$ 
8:   end for
9: end if

```

design eliminates. WISEConv resolves this tension through offline calibration combined with a sync-free online feedback loop.

Offline calibration. The activity range is discretized into N_ℓ levels (we use $N_\ell = 10$, spanning density from 100% to ~10%). For each convolution shape in the model and each activity level, a one-time calibration pass generates a synthetic worklist of the corresponding length, sweeps all candidate tile configurations and split-K values, and records the fastest combination. The result is a per-shape, per-level lookup table mapping activity to the optimal (B_M, B_N, B_K, S_k) . This table is computed once at deployment and persisted to disk; at inference time, configuration selection reduces to a single table lookup indexed by the current activity level.

Sync-free activity feedback. WISEConv delivers the current activity level to the host without stalling the compute stream via an asynchronous readback on a dedicated copy stream. After each frame's construction writes the active count to device memory, the runtime enqueues an asynchronous device-to-host transfer of that count, gated by an event so the copy waits until construction completes. A ring buffer of N_s slots ($N_s = 4$) absorbs variation in transfer latency; if all slots are in flight, the current frame's count is dropped without blocking.

At the start of the next frame, completed slots are polled without synchronization. The most recent available count maps to the nearest calibrated activity level via binary search, and each convolution module's tile configuration is updated accordingly.

Algorithm 3 summarizes the per-frame feedback loop. The applied configuration is lagged by at least one frame relative to the current input. This lag does not affect correctness: the configuration determines only the parallel decomposition, not the computed values. In streaming video, scene changes are dominated by continuous object motion and smooth illumination shifts, so frame-to-frame activity variation is small relative to the calibration bin width. The lagged configuration therefore remains near-optimal; §5 quantifies this empirically.

Design rationale. Two alternatives fail under the constraints of streaming inference. A fixed configuration forces a single compromise: over-decomposing at high activity (paying unnecessary atomic overhead) or under-utilizing the device at low activity. Per-layer host-side selection requires a device-to-host transfer and synchronization for each layer, stalling the compute pipeline proportionally to layer count. Offline calibration absorbs the profiling cost once; the async ring-buffer readback provides frame-level adaptation with zero compute-stream interference, maintaining T across the full activity range.

4 Implementation

We implement WISEConv as a CUDA C++ extension for PyTorch. The current backend executes FP32 tensors in channels-last layout and provides the mask-aware operators required by the evaluated networks: convolution, transposed convolution, pooling, pointwise activation, addition, concatenation, and upsampling. A model-conversion pass replaces compatible PyTorch modules while preserving the graph topology, convolution parameters, and learned weights. The upstream application continues to generate masks.

The construction kernel assigns contiguous output positions to each warp. A thread tests one receptive field and terminates once it finds an active input. A warp ballot identifies active lanes, population counts give their local ranks, and one atomic operation reserves a contiguous worklist segment for the warp. Active lanes then write their coordinates in lane order. This realizes the tile-local ordering in Section 3.2 without a device-wide prefix scan or sorting pass. An identity 1×1 convolution has the same input and output coordinate space; when its input already carries compatible mask and worklist metadata, the operator reuses that worklist and skips construction.

The compute kernel uses a persistent, tiled implicit-GEMM organization. It keeps partial sums in registers and stages input and weight tiles through shared memory. Supported architectures use asynchronous global-to-shared copies, with a synchronous path for earlier GPUs. Split- K exposes additional parallel work when the active-coordinate dimension alone cannot occupy the device. Transposed convolution follows the same output-driven organization, but partitions output coordinates by stride phase so that each phase visits only its valid kernel offsets.

Efficient scheduling depends jointly on the GPU, layer shape, and activity. We therefore autotune the sparse tile shape and split- K factor and cache the choice by device, convolution parameters, activity level, and execution policy. For layers whose state-update semantics permit dense evaluation, the candidate set can also include a cuDNN path. A sparse-only policy isolates the worklist-driven kernel, and the cache key distinguishes the two policies and the requested determinism mode. Autotuning and one-time weight

preparation complete before the timed sequence described in Section 5.1.

For fixed input values, weights, and mask, WISEConv evaluates Eq. (2) at every active output coordinate. Floating-point reordering may produce small numerical differences from cuDNN, so we verify active outputs with numerical tolerances rather than requiring bitwise identity. This check separates backend correctness from the task-quality effect of the fixed upstream mask policy.

5 Evaluation

5.1 Experimental Setup

We evaluate WISEConv across four perception models, three mask sources, four GPU platforms, and three competing execution paradigms. The evaluation asks four questions: (1) does WISEConv reduce full-model latency relative to the fastest available path, (2) how effectively does it convert required-work reduction into latency reduction, (3) what overhead does worklist construction add, and (4) does replacing the masked-convolution backend change task output or accuracy?

5.1.1 Platforms. We evaluate four representative GPU platforms: two embedded SoCs (Jetson Xavier NX and Jetson AGX Orin), a laptop-class mobile GPU (RTX 4070 Laptop), and a desktop GPU (RTX 3080). These devices cover the hardware classes commonly used for onboard deployment and robotics development. The two Jetson systems anchor our target deployment setting, while the discrete GPUs test whether WISEConv generalizes beyond embedded SoCs.

5.1.2 Models, Tasks, and Mask Sources. Our selected workloads span three robotic perception tasks:

- **Event-based optical flow.** We evaluate FireFlowNet [25], a lightweight event-based optical-flow model that delivers high inference rates while retaining competitive accuracy, on MVSEC [42]. MVSEC contains event-camera sequences and corresponding ground-truth flow from indoor quadrotor flight and outdoor driving. Following Ev-Conv [7], consecutive 50-ms windows advance by 1 ms; events entering or leaving the window form the sparse increment from which we derive layer masks. We calibrate the sparsification policy on indoor_flying1 and evaluate on the other three sequences.
- **Video object detection.** We evaluate YOLOv8n and YOLOv8m [18] on MOT16 [23], a collection of crowded pedestrian videos with annotated person boxes. Both models produce bounding boxes and confidence scores, while their different scales expose how model size affects sparse-execution efficiency. DeltaCNN [27] supplies the temporal mask mechanism by propagating thresholded activation deltas between successive frames.

- **Human pose estimation.** We use the DynConv pose model [34] on MPII [1], which depicts people performing diverse activities and annotates their 2D body joints. The model predicts joint heatmaps; its learned spatial gates provide input-dependent masks at gated blocks.

Table 1 summarizes their models and evaluation scales. We use official weights for all four models [18, 25, 34]. For each workload, we fix these weights, inputs, and upstream mask policy; the sparse paths receive the same resulting layer masks and differ only in their masked-convolution backend.

Table 1. Evaluation workloads.

	FireFlowNet [25]	YOLOv8n/m [18]	DynConv Pose [34]
Task	Optical flow	Detection	Human pose
Mask source	Ev-Conv [7]	DeltaCNN [27]	DynConv [34]
Dataset	MVSEC [42]	MOT16 [23]	MPII [1]
Input size	256×256	640×640	256×256
Parameters	0.056M	3.16M / 25.9M	6.89M
Eval. samples	196.8K	2,575	2,958

5.1.3 Execution Baselines. We compare WISEConv against three FP32 execution paths established by prior work.

- **Dense** uses the native PyTorch/cuDNN backend [5] with sparse execution disabled.
- **Tile skipping** follows the tile-sparse execution established by SBNNet, Ev-Conv, and DeltaCNN [7, 27, 29].
- **Gather-scatter** uses the original DynConv CUDA backend for human pose [34] and an implementation equivalent to MMCV MaskedConv2d for FireFlowNet and YOLO [4].

5.1.4 Metrics and Methodology. Our primary metric is full-model GPU latency. The timed region includes mask propagation, worklist construction, all network operators, and output update, but excludes host-to-device input transfer, calibration, autotuning, and the one-time dense state initialization required by incremental models. We use CUDA events on the model’s compute stream and process complete held-out sequences rather than isolated synthetic frames. Each sequence is repeated three times; we retain per-frame samples and report both aggregate latency and dispersion.

To explain latency, we report the required-work ratio ρ_{work} , useful-compute ratio η , convolution throughput, and the construction fraction of operator latency. Because layers differ in cost, an unweighted average of layer activities misstates the work a mask removes, so we weight each layer by its dense MAC count,

$$\rho_{\text{work}} = \frac{\sum_l \rho^l W_{\text{dense}}^l}{\sum_l W_{\text{dense}}^l}, \quad \rho^l = \frac{\|M^l\|_0}{H_{\text{out}}^l W_{\text{out}}^l}, \quad (5)$$

where W_{dense}^l is the dense MAC count of layer l . Task quality is measured with Average Endpoint Error (AEE) for optical flow, detection AP against the fixed evaluation references for YOLO, and PCKh for human pose. Accuracy comparisons use the same masks and operating point as latency measurements; they evaluate the upstream policy, while operator correctness is checked separately against dense convolution at every requested output.

5.2 Full-Model Latency

5.3 Efficiency Analysis

5.4 Construction Overhead

5.5 Task Accuracy

References

- [1] Mykhaylo Andriluka, Leonid Pishchulin, Peter Gehler, and Bernt Schiele. 2014. 2D Human Pose Estimation: New Benchmark and State of the Art Analysis. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3686–3693.
- [2] Rik J. Bouwmeester, Federico Paredes-Vallés, and Guido C. H. E. de Croon. 2023. NanoFlowNet: Real-time Dense Optical Flow on a Nano Quadcopter. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 1996–2003.
- [3] Lukas Cavigelli, Philippe Degen, and Luca Benini. 2017. CBInfer: Change-Based Inference for Convolutional Neural Networks on Video Data. In *Proc. Int. Conf. Distrib. Smart Cameras (ICDSC)*. 1–8.
- [4] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, Zheng Zhang, Dazhi Cheng, Chenchen Zhu, Tianheng Cheng, Qijie Zhao, Buyu Li, Xin Lu, Rui Zhu, Yue Wu, Jifeng Dai, Jingdong Wang, Jianping Shi, Wanli Ouyang, Chen Change Loy, and Dahua Lin. 2019. MMDetection: Open MMLab Detection Toolbox and Benchmark. *CoRR* abs/1906.07155 (2019).
- [5] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. *CoRR* abs/1410.0759 (2014).
- [6] Christopher B. Choy, JunYoung Gwak, and Silvio Savarese. 2019. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [7] Sankeerth Durvasula, Yushi Guan, and Nandita Vijaykumar. 2023. Ev-Conv: Fast CNN Inference on Event Camera Inputs for High-Speed Robot Perception. *IEEE Robotics Autom. Lett.* 8, 6 (2023), 3174–3181.
- [8] Davide Falanga, Philipp Foehn, Peng Lu, and Davide Scaramuzza. 2018. PAMPC: Perception-Aware Model Predictive Control for Quadrotors. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2018, Madrid, Spain, October 1-5, 2018*. IEEE, 1–8. doi:10.1109/IROS.2018.8593739
- [9] Davide Falanga, Suseong Kim, and Davide Scaramuzza. 2019. How Fast Is Too Fast? The Role of Perception Latency in High-Speed Sense and Avoid. *IEEE Robotics Autom. Lett.* 4, 2 (2019), 1884–1891. doi:10.1109/LRA.2019.2898117
- [10] Davide Falanga, Kevin Kleber, and Davide Scaramuzza. 2020. Dynamic obstacle avoidance for quadrotors with event cameras. *Sci. Robotics* 5, 40 (2020), 9712. doi:10.1126/SCIROBOTICS.AAZ9712
- [11] Ruibo Fan, Wei Wang, and Xiaowen Chu. 2024. DTC-SpMM: Bridging the Gap in Accelerating General Sparse Matrix Multiplication with Tensor Cores. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024– 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafirir (Eds.). ACM, 253–267. doi:10.1145/3620666.3651378

- [12] Michael Figurnov, Maxwell D. Collins, Yukun Zhu, Li Zhang, Jonathan Huang, Dmitry P. Vetrov, and Ruslan Salakhutdinov. 2017. Spatially Adaptive Computation Time for Residual Networks. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 1790–1799.
- [13] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU Kernels for Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
- [14] Yizhao Gao, Baoheng Zhang, Xiaojuan Qi, and Hayden Kwok-Hay So. 2023. DPACS: Hardware Accelerated Dynamic Neural Network Pruning through Algorithm-Architecture Co-design. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25–29, 2023*, Tor M. Aamodt, Natalie D. Enright Jerger, and Michael M. Swift (Eds.). ACM, 237–251. doi:10.1145/3575693.3575728
- [15] Benjamin Graham, Martin Engelcke, and Laurens van der Maaten. 2018. 3D Semantic Segmentation with Submanifold Sparse Convolutional Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [16] Amirhossein Habibian, Davide Abati, Taco S. Cohen, and Babak Ehteshami Bejnordi. 2021. Skip-Convolutions for Efficient Video Processing. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2695–2704.
- [17] Drew Hanover, Antonio Loquercio, Leonard Bauersfeld, Angel Romero, Robert Penicka, Yunlong Song, Giovanni Cioffi, Elia Kaufmann, and Davide Scaramuzza. 2024. Autonomous Drone Racing: A Survey. *IEEE Trans. Robotics* 40 (2024), 3044–3067. doi:10.1109/TRO.2024.3400838
- [18] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. 2023. Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>
- [19] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman P. Amarasinghe. 2017. The Tensor Algebra Compiler. *Proc. ACM Program. Lang.* 1, OOPSLA (2017).
- [20] Fanrong Li, Gang Li, Xiangyu He, and Jian Cheng. 2021. Dynamic Dual Gating Neural Networks. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*. 5310–5319.
- [21] Renyuan Liu, Yuyang Leng, Shilei Tian, Shaohan Hu, Chun-Fu (Richard) Chen, and Shuochao Yao. 2024. DynaSpa: Exploiting Spatial Sparsity for Efficient Dynamic DNN Inference on Devices. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems, SenSys 2024, Hangzhou, China, November 4–7, 2024*, ACM, 422–435. doi:10.1145/3666025.3699348
- [22] Nico Messikommer, Daniel Gehrig, Antonio Loquercio, and Davide Scaramuzza. 2020. Event-Based Asynchronous Sparse Convolutional Networks. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 415–431.
- [23] Anton Milan, Laura Leal-Taixé, Ian D. Reid, Stefan Roth, and Konrad Schindler. 2016. MOT16: A Benchmark for Multi-Object Tracking. *CoRR abs/1603.00831* (2016).
- [24] Asit K. Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. 2021. Accelerating Sparse Deep Neural Networks. *CoRR abs/2104.08378* (2021).
- [25] Federico Paredes-Vallés and Guido C. H. E. de Croon. 2021. Back to Event Basics: Self-Supervised Learning of Image Reconstruction for Event Cameras via Photometric Constancy. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3445–3454.
- [26] Mathias Parger, Chengcheng Tang, Thomas Neff, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2023. MotionDeltaCNN: Sparse CNN Inference of Frame Differences in Moving Camera Videos with Spherical Buffers and Padded Convolutions. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1–6, 2023*. 17246–17255.
- [27] Mathias Parger, Chengcheng Tang, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2022. DeltaCNN: End-to-End CNN Inference of Sparse Frame Differences in Videos. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 12487–12496.
- [28] Hongwu Peng, Xi Xie, Kaustubh Shivdikar, Md Amit Hasan, Jiahui Zhao, Shaoyi Huang, Omer Khan, David R. Kaeli, and Caiwen Ding. 2024. MaxK-GNN: Extremely Fast GPU Kernel Design for Accelerating Graph Neural Networks Training. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024– 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafir (Eds.). ACM, 683–698. doi:10.1145/3620665.3640426
- [29] Mengye Ren, Andrei Pokrovsky, Bin Yang, and Raquel Urtasun. 2018. SBNet: Sparse Blocks Network for Fast Inference. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 8711–8720.
- [30] Spconv Contributors. 2022. Spconv: Spatially Sparse Convolution Library. <https://github.com/traveller59/spconv>.
- [31] Martin Sundermeyer, Arsalan Mousavian, Rudolph Triebel, and Dieter Fox. 2021. Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 13438–13444.
- [32] Haotian Tang, Zhijian Liu, Xiuyu Li, Yujun Lin, and Song Han. 2022. TorchSparse: Efficient Point Cloud Inference Engine. In *Proceedings of Machine Learning and Systems (MLSys)*.
- [33] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. TorchSparse++: Efficient Training and Inference Framework for Sparse Convolution on GPUs. In *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*. 225–239.
- [34] Thomas Verelst and Tinne Tuytelaars. 2020. Dynamic Convolutions: Exploiting Spatial Sparsity for Faster Inference. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2320–2329.
- [35] Chen Wang, Danfei Xu, Yuke Zhu, Roberto Martin Martin, Cewu Lu, Li Fei-Fei, and Silvio Savarese. 2019. DenseFusion: 6D Object Pose Estimation by Iterative Dense Fusion. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3343–3352.
- [36] Kunyun Wang, Shuo Yang, Jieru Zhao, Wenchao Ding, Quan Chen, Jingwen Leng, and Minyi Guo. 2025. SparseTem: Boosting the Efficiency of CNN-Based Video Encoders by Exploiting Temporal Continuity. In *Advanced Parallel Processing Technologies - 16th International Symposium, APPT 2025, Athens, Greece, July 13–16, 2025, Proceedings*. 246–256.
- [37] Diana Wofk, Fangchang Ma, Tien-Ju Yang, Sertac Karaman, and Vivienne Sze. 2019. FastDepth: Fast Monocular Depth Estimation on Embedded Systems. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 6101–6108.
- [38] Zhenda Xie, Zheng Zhang, Xizhou Zhu, Gao Huang, and Stephen Lin. 2020. Spatially Adaptive Inference with Stochastic Feature Sampling and Interpolation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 531–548.
- [39] Jiacheng Yang, Christina Giannoula, Jun Wu, Mostafa Elhoushi, James Gleeson, and Gennady Pekhimenko. 2024. Minuet: Accelerating 3D Sparse Convolutions on GPUs. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22–25, 2024*. 786–802.
- [40] Zihao Ye, Ruihang Lai, Junru Shao, Tianqi Chen, and Luis Ceze. 2023. SparseTIR: Composable Abstractions for Sparse Compilation in Deep Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [41] Changqian Yu, Jingbo Wang, Chao Peng, Changxin Gao, Gang Yu, and Nong Sang. 2018. BiSeNet: Bilateral Segmentation Network for Real-Time Semantic Segmentation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 334–349.

991	[42] Alex Zihao Zhu, Dinesh Thakur, Tolga Özaslan, Bernd Pfrommer,	Event Camera Dataset: An Event Camera Dataset for 3D Perception.	1046
992	Vijay Kumar, and Kostas Daniilidis. 2018. The Multi Vehicle Stereo	<i>CoRR</i> abs/1801.10202 (2018).	1047
993			1048
994			1049
995			1050
996			1051
997			1052
998			1053
999			1054
1000			1055
1001			1056
1002			1057
1003			1058
1004			1059
1005			1060
1006			1061
1007			1062
1008			1063
1009			1064
1010			1065
1011			1066
1012			1067
1013			1068
1014			1069
1015			1070
1016			1071
1017			1072
1018			1073
1019			1074
1020			1075
1021			1076
1022			1077
1023			1078
1024			1079
1025			1080
1026			1081
1027			1082
1028			1083
1029			1084
1030			1085
1031			1086
1032			1087
1033			1088
1034			1089
1035			1090
1036			1091
1037			1092
1038			1093
1039			1094
1040			1095
1041			1096
1042			1097
1043			1098
1044			1099
1045			1100